
Assignment 2: Q-Learning - Tabular & Deep

Anh Trinh
s4870085

Muhamad Iqbal Arsa
s4859049

Abstract

The CartPole environment presents a high-dimensional continuous state space that makes traditional tabular reinforcement learning methods impractical due to the curse of dimensionality. Training stability in this domain is fundamentally challenged by the deadly triad, the divergence caused by function approximation, bootstrapping, and off-policy learning, which further exacerbated by the temporal correlations between samples. To counteract these instabilities, a Deep Q-Network (DQN) architecture is implemented, utilizing an experience replay buffer to break temporal correlations and a separate target network utilizing infrequent weight updates to provide a stable target for the loss function. While experimental results confirm that this integrated approach significantly accelerates learning and achieves peak episode returns of ≈ 430 , the model remains unable to maintain stable convergence at the environment's maximum theoretical limit of 500.

1. Introduction

The main challenge of the CartPole environment is its continuous state space, which makes the number of possible states extremely large (Towers et al., 2024). As a result, traditional tabular methods become impractical due to the curse of dimensionality. Deep Reinforcement Learning (DRL) addresses this limitation by using neural networks as function approximators to estimate value functions.

However, combining neural networks with standard Q-learning introduces training instability, caused by the interaction of function approximation, bootstrapping, and off-policy learning, commonly known as the deadly triad (Plaat, 2022). To mitigate this issue, Deep Q-Networks (DQN) introduce two key techniques: Experience Replay (ER) and Target Networks (TN).

In this report, we study the CartPole environment by first implementing a naive DQN approach using a neural network without ER and TN, and performing an ablation study on several parameters. We then evaluate four variants: naive DQN, DQN with only ER, DQN with only TN, and the full

DQN, to analyze how each component contributes to the overall performance.

2. Theory

2.1. Derivation of the DQN Loss Function

In deep Q-learning, the loss function is derived directly from the standard tabular Q-learning update formula (Plaat, 2022). It is constructed by combining the bootstrapping principle from reinforcement learning with loss function minimization techniques used in deep learning.

In particular, the loss function is calculated as the squared difference between the network's current prediction and the update target (Mnih et al., 2015). This update target y is defined as the immediate reward plus the discounted maximum Q-value of the next state, computed using a separate target network with parameters θ^- :

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a') \quad (1)$$

The loss function $L(\theta)$ aims to minimize this squared error over sampled transitions from an experience replay buffer D :

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} [(y - Q_{\theta}(s,a))^2] \quad (2)$$

An important distinction from supervised learning is that the Q-learning loss function minimizes a moving target that depends on the previous parameter θ^- , therefore the target shifts accordingly as the network's weights move (Plaat, 2022).

2.2. Limitations of Tabular Updates

Tabular method relies on storing exact values of individual states in a lookup table. This method fails when the dimensions of a problem increases, as the number of unique states grows exponentially (Plaat, 2022). For environments with continuous state spaces, such as the CartPole, a discrete lookup table is not practical, as it would require arbitrary discretization. Moreover, the lookup table size increases exponentially, it becomes practically impossible to experience and update every state. Therefore, instead of tabular update, we use function approximation and deep neural networks to generalize learning in a massive state space.

2.3. Pseudo Code for DQN with ER and TN

Algorithm 1 shows the DQN training process as follow (Mnih et al., 2015). To make learning more stable, the agent stores past experiences in a replay buffer and samples random mini-batches. This helps reduce the correlation between consecutive data (Lin, 1992; Riedmiller, 2005). Furthermore, a separate target network (θ^-) is used to compute the Bellman target y , providing a stationary target that is updated periodically to prevent training instability and divergence (Mnih et al., 2015; Sutton & Barto, 2020).

Algorithm 1 Deep Q-Learning (DQN) Training

- 1: **Initialize** policy network Q_θ , target network Q_{θ^-} with weights $\theta^- = \theta$, and replay buffer D
 - 2: **for** each training step $t = 1, 2, \dots, T$ **do**
 - 3: Observe current state s (reset environment if previous state was terminal)
 - 4: Select action a via ϵ -greedy policy based on $Q_\theta(s)$
 - 5: Execute action a , observe reward r , next state s' , and terminal flag
 - 6: Store transition $(s, a, r, s', \text{terminal})$ in D
 - 7: Sample random mini-batch of transitions from D
 - 8: Compute target $y = r$ if terminal, otherwise $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$
 - 9: Update θ by minimizing loss $\mathcal{L} = (y - Q_\theta(s, a))^2$
 - 10: Periodically sync target network: $\theta^- \leftarrow \theta$ and decay ϵ
 - 11: **end for**
-

3. Experiments

3.1. Setup

This experiment evaluates a DQN agent in the CartPole environment (Towers et al., 2024), where the goal is to balance a pole on a moving cart by choosing left or right actions. The state space consists of four continuous variables: cart position, cart velocity, pole angle, and pole angular velocity. The agent receives a reward of +1 for each step the pole remains upright, with a maximum return of 500 per episode.

The experiments were run for 10^6 environment steps and repeated 5 times to ensure reliable results. For visualization, the learning curves were smoothed using a moving average over 200 episodes. For all network architectures, we used a single hidden layer, which is sufficient for the CartPole environment. According to the Universal Approximation Theorem, a neural network with a single hidden layer can approximate any continuous function to any degree of accuracy (Sutton & Barto, 2020). The full list of fixed hyperparameters is shown in Appendix A Table 1. In the ablation study, only the network size, learning rate, exploration factor, and update-to-data ratio (train frequency)

were changed, while all other parameters were kept the same.

3.2. Implementation Q-Learning in Cartpole

We implement Naive DQN, an agent that utilizes a neural network without experience replay and a target network. Figure 1 shows a positive trend in episode returns over 10^6 steps, with initial progress during the first 200k steps. However, instead of converging, scores fluctuated noticeably between 100 and 300 over the next 500k steps. Although returns peaked at 400 after 800k steps, the agent failed to reach and maintain the CartPole’s maximal score of 500.

These sharp fluctuations in Naive DQN and the inability to achieve a stable optimum are caused by the absence of ER and TN. As demonstrated by (Mnih et al., 2015), neural networks rarely perform well without additional modification in reinforcement learning because learning directly from sequential steps leads to highly correlated data, while optimizing a rapidly changing Bellman target makes the training process unstable (Plaat, 2022; Mnih et al., 2015).

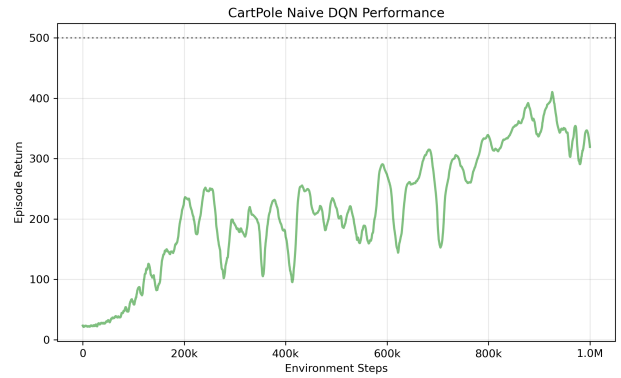


Figure 1. Learning curves for Naive DQN in CartPole

3.3. Hyperparameter Ablation Study

3.3.1. LEARNING RATE

As presented in Figure 2, applying a low learning rate (10^{-4}) results in the agent learning very slowly, where it needs 750k steps just to exceed the return of 100. In contrast, a high learning rate (10^{-2}) allows rapid learning in the early phases, but experienced performance drops after 700k steps, likely due to large updates that cause instability and overshooting of optimal values (Plaat, 2022). A moderate learning rate (10^{-3}) performs well, giving optimal balance, the return reward demonstrates a positive trend and successfully surpasses the higher learning rate at 750k steps until the end of training.



Figure 2. The effect of different learning rates on the training performance of Naive DQN in the CartPole environment



Figure 4. The effect of different Update-to-data Ratio on the training performance of Naive DQN in the CartPole environment

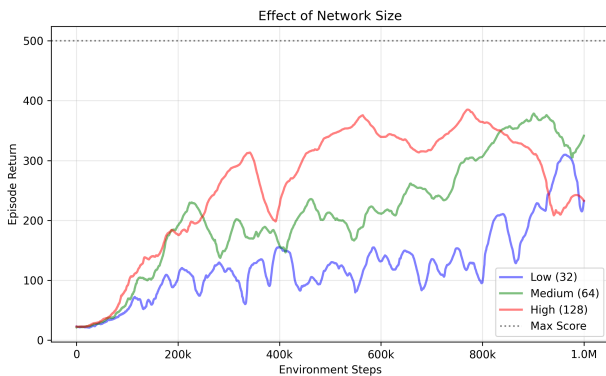


Figure 3. The effect of different Network Size on the training performance of Naive DQN in the CartPole environment

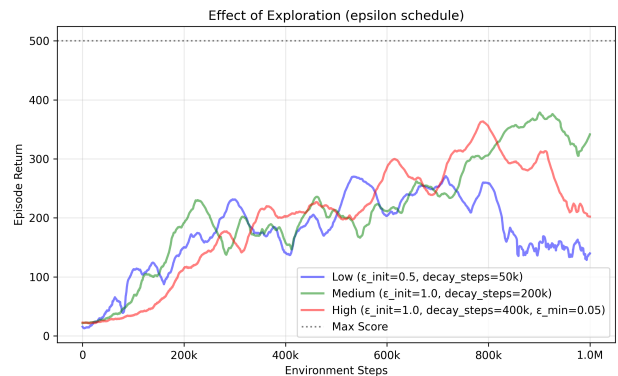


Figure 5. The effect of different Exploration Rate on the training performance of Naive DQN in the CartPole environment

3.3.2. NETWORK SIZE

As shown in Figure 3, using a small network (size 32) leads to very slow learning progress, indicating underfitting due to limited capacity. The agent struggles and stays below a return of 150 during the first 800k steps. In contrast, the large network (size 128) learns quickly at the beginning, but its performance drops sharply after 800k steps, suggesting overfitting, which becomes worse in Naive DQN because the agent learns from highly correlated sequential data without using an ER buffer (Mnih et al., 2015). The medium network (size 64) shows a better balance, with steady and stable improvement throughout the training process.

3.3.3. UPDATE-TO-DATA RATIO

As seen in Figure 4, the update-to-data ratio significantly impacts the learning speed and stability of the naive deep Q-learning approach. Setting a high interval (8 steps) causes the agent to learn quite slowly, struggling to break a return of 250 throughout the training process. On the other hand, updating the parameters too frequently (every 1 step) helps with early learning but ultimately causes the network to become unstable, with performance dropping after 800k steps.

The medium setting (4 steps) strikes an effective balance. While it starts slower, it maintains a steady and positive learning trend, outperforming the other configurations by the end of the run.

3.3.4. EXPLORATION RATE

As shown in Figure 5, the exploration schedule is a critical factor in the agent's performance. A prolonged exploration phase (decaying over 400k steps) holds the agent back, keeping returns low as it struggles to efficiently exploit what it has learned. On the other hand, a rapid decay (50k steps) forces the agent to commit too early. It learns fast at the beginning but suffers a sharp drop after 500k steps because it falls into a specialization trap and gets stuck in local optima (Plaat, 2022). The medium schedule (200k steps) provides the best middle ground. It allows the agent enough time to achieve sufficient state space coverage in the CartPole environment before shifting to exploitation, resulting in the most stable overall convergence.

3.3.5. OPTIMAL CONFIGURATION SELECTION

Based on the results of the ablation study, the optimal configuration for the CartPole environment was determined as

follows: a **learning rate** of 10^{-3} , a **network size** of 64 nodes, an **update-to-data ratio** of 4, and an **exploration factor** defined by $\epsilon_{\text{init}} = 1.0$ with a decay duration of 200k steps. These values will be used for the next experiment involving ER and TN.

3.4. Target Network and Experience Replay Comparison

To evaluate the impact of mitigating the effects of the deadly triad, Figure 6 compares four Q-learning configurations over 10^6 steps: Naive, Only TN, Only ER, and Full DQN, alongside a provided baseline.

The Full DQN demonstrates the fastest initial learning, peaking at an episode return of ~ 430 near 200k steps. This synergy is achieved by combining two distinct methods. Adding Experience Replay (ER) gives the agent a replay buffer of randomly sampled training states (Plaatt, 2022). Adding Target Network (TN) delays the weight updates, providing a stable target to evaluate the samples from replay buffer against (Plaatt, 2022).

On its own, the Only ER model mirrors this early speed but plateaus at a lower value (~ 380), confirming that breaking temporal correlations between samples is the primary driver for early learning. Conversely, the Only TN configuration fails drastically, rarely exceeding 300 return. Without experience replay to break correlations, the stable targets are insufficient to prevent divergence.

Interestingly, while the Naive approach starts very slowly, it steadily catches up to the Only ER model by 900k steps. However, despite these algorithmic improvements, none of the models achieve stable, long-term convergence at environment's maximum score of 500. In contrast, the provided baseline successfully reaches and largely maintains this optimal performance after approximately 500k steps.



Figure 6. Performance comparison of Naive Q-Learning, Only Target Network, Only Experience Replay, and Full DQN architectures in CartPole.

4. Discussion

The comparison between configurations highlights the distinct roles of ER and TN in mitigating the deadly triad. The Naive DQN suffered severe instability from correlated data and shifting targets. While the Naive DQN still theoretically has the capacity to learn, as evidenced by it gradually catching up near the end of the 10^6 steps, it is very inefficient. This catch up could simply represent the network brute-forcing its way to a sub-optimal policy. Ultimately, it requires almost five times more interaction with the environment to achieve the same score as the full DQN, highlighting how ER and TN are essential not just for convergence, but for stabilizing the initial learning curve and ensuring efficiency.

By breaking temporal correlations, the Only ER model learned quickly but plateaued. On the other hand, the Only TN model diverged, proving that stationary targets require decorrelated data. Using both mechanisms allowed the Full DQN to achieve the fastest initial convergence. However the full DQN's performance drops after 800k steps and fails to maintain the optimal score of 500 might because of over training periods. The use of the *max* operator in the Bellman equation, along with noise from function approximation can lead the agent to gradually build up positive errors over time, which may damage the learned policy (Thrun & Schwartz, 1993). Its unable to maintain the optimal score, showing that these mechanisms (ER and TN) cannot fully eliminate training instabilities in CartPole.

5. Conclusion

This study successfully applied Deep Q-Networks to the CartPole environment, overcoming the limitations of tabular methods in continuous state spaces. As illustrated in Figure 6, the Full DQN configuration provided superior performance by combining experience replay to break temporal correlations and infrequent weight updates to improve the stability of the Q-targets. Despite these improvements, a notable weakness in the current experiment setup is the inability to achieve stable, long-term convergence at the maximum score of 500. The observed fluctuations near the end of training suggest that the deadly triad remains a challenge.

To address these limitations, future work should implement Double DQN to prevent Q-value overestimation or Dueling DDQN, which computes an advantage function to standardize values on a baseline for the actions of a state. Furthermore, investigating Prioritized Experience Replay could improve learning efficiency by replaying important actions based on their absolute TD error, potentially stabilizing the agent's performance in the final training phases.

References

- Lin, L.-J. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015. doi: 10.1038/nature14236.
- Plaat, A. *Deep Reinforcement Learning*. Springer Nature Singapore Pte Ltd., 2022. doi: 10.1007/978-981-19-0638-1. URL <https://doi.org/10.1007/978-981-19-0638-1>.
- Riedmiller, M. Neural fitted q iteration – first experiences with a data efficient neural reinforcement learning method. In Gama, J., Camacho, R., Brazdil, P. B., Jorge, A. M., and Torgo, L. (eds.), *Machine Learning: ECML 2005*, volume 3720 of *Lecture Notes in Computer Science*, pp. 317–328, Berlin, Heidelberg, 2005. Springer.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2 edition, 2020.
- Thrun, S. and Schwartz, A. Issues in using function approximation for reinforcement learning. In *Proceedings of the Fourth Connectionist Models Summer School*, Hillsdale, NJ, 1993. Lawrence Erlbaum.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulao, M., Kallinteris, A., Krimmel, M., KG, A., et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.

A. Appendix

Table 1. Global Hyperparameter Settings and Ablation Ranges. Values in brackets represent the [Low, Medium, High] settings used in the study

Category	Hyperparameter	Value(s)	Description
Ablation Study	Learning Rate (α)	$[10^{-4}, 10^{-3}, 10^{-2}]$	Controls the step size of weight updates.
	Network Size	[32, 64, 128]	Determines the model's capacity.
	Train Frequency	[1, 4, 8]	Number of environment steps between updates.
	Exploration Factor (Decay Steps)	[50k, 200k, 400k]	Duration of exploration-to-exploitation decay.
Fixed Parameters	Discount Factor (γ)	0.99	Importance of future vs immediate rewards.
	Network Sync Rate	100	Interval for updating the Target Network.
	Replay Memory Size	10,000	Max number of past transitions in buffer.
	Mini Batch Size	64	Samples used for a single training update.
	Epsilon Init / Min	1.0 / 0.01	Probability range for taking a random action.
Environment	Environment	CartPole-v1	Balancing a pole on a friction-less track.
	Max Steps per Run	10^6	Total interaction budget for training.